

```
package leave.Management;

import java.sql.*;

import java.util.Scanner;


public class LeaveManagement {

    static Scanner sc = new Scanner(System.in);


    static final String URL =

        "jdbc:mysql://localhost:3306/jdbc_project";

    static final String USER = "root";

    static final String PASS = "Shalu@1302";


    public static void main(String[] args) {

        try {

            Class.forName("com.mysql.cj.jdbc.Driver");

            Connection con =

                DriverManager.getConnection(URL, USER, PASS);

            System.out.println("Database connected successfully!");

            createTables(con);

            int choice;

            do {

                System.out.println("\n===== EMPLOYEE LEAVE MANAGEMENT =====");
```

```
System.out.println("1. Employee Registration");  
System.out.println("2. Display Employees");  
System.out.println("3. Update Employee");  
System.out.println("4. Delete Employee");  
System.out.println("5. Apply Leave");  
System.out.println("6. Display Leave Requests");  
System.out.println("7. Approve / Reject Leave");  
System.out.println("8. Leave Balance");  
System.out.println("9. Exit");
```

```
System.out.print("Enter choice: ");  
choice = Integer.parseInt(sc.nextLine());
```

```
switch (choice) {
```

```
    case 1:
```

```
        addEmployee(con);
```

```
        break;
```

```
    case 2:
```

```
        displayEmployees(con);
```

```
        break;
```

```
    case 3:
```

```
        updateEmployee(con);
```

```
        break;
```

```
    case 4:
```

```
deleteEmployee(con);
```

```
break;
```

```
case 5:
```

```
applyLeave(con);
```

```
break;
```

```
case 6:
```

```
displayLeaveRequests(con);
```

```
break;
```

```
case 7:
```

```
approveRejectLeave(con);
```

```
break;
```

```
case 8:
```

```
displayBalance(con);
```

```
break;
```

```
case 9:
```

```
System.out.println("Thank you!");
```

```
break;
```

```
default:
```

```
System.out.println("Invalid choice!");
```

```
}
```

```
} while (choice != 9);
```

```
        con.close();

    } catch (Exception e) {
        e.printStackTrace();
    }
}

// CREATE TABLES

static void createTables(Connection con) {

    try {
        Statement st = con.createStatement();

        st.executeUpdate(
            "CREATE TABLE IF NOT EXISTS employees (" +
            "emp_id INT PRIMARY KEY AUTO_INCREMENT," +
            "emp_name VARCHAR(50)," +
            "department VARCHAR(50)," +
            "email VARCHAR(100))"
        );

        st.executeUpdate(
            "CREATE TABLE IF NOT EXISTS leave_requests (" +
            "leave_id INT PRIMARY KEY AUTO_INCREMENT," +
            "emp_id INT," +
```

```

        "leave_type VARCHAR(30)," +
        "start_date DATE," +
        "end_date DATE," +
        "reason VARCHAR(100)," +
        "status VARCHAR(20) DEFAULT 'Pending'," +
        "FOREIGN KEY(emp_id) REFERENCES employees(emp_id))"
    );

```

```

st.executeUpdate(
    "CREATE TABLE IF NOT EXISTS leave_balance (" +
    "emp_id INT PRIMARY KEY," +
    "total_leaves INT DEFAULT 20," +
    "used_leaves INT DEFAULT 0," +
    "remaining_leaves INT DEFAULT 20," +
    "FOREIGN KEY(emp_id) REFERENCES employees(emp_id))"
);

```

```

System.out.println("Tables created successfully!");

```

```

    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

```

// 1. EMPLOYEE REGISTRATION - CREATE

```

```

static void addEmployee(Connection con) {

```

```
try {  
    System.out.print("Enter employee name: ");  
    String name = sc.nextLine();  
  
    System.out.print("Enter department: ");  
    String dept = sc.nextLine();  
  
    System.out.print("Enter email: ");  
    String email = sc.nextLine();  
  
    String sql =  
        "INSERT INTO employees(emp_name, department, email) " +  
        "VALUES(?,?,?)";  
  
    PreparedStatement ps =  
        con.prepareStatement(  
            sql, Statement.RETURN_GENERATED_KEYS);  
  
    ps.setString(1, name);  
    ps.setString(2, dept);  
    ps.setString(3, email);  
  
    ps.executeUpdate();  
  
    ResultSet rs = ps.getGeneratedKeys();  
  
    if (rs.next()) {
```

```
int id = rs.getInt(1);
```

```
PreparedStatement balance =
```

```
con.prepareStatement(
```

```
"INSERT INTO leave_balance(emp_id) VALUES(?)");
```

```
balance.setInt(1, id);
```

```
balance.executeUpdate();
```

```
System.out.println(
```

```
"Employee registered successfully!");
```

```
System.out.println("Employee ID: " + id);
```

```
}
```

```
} catch (SQLException e) {
```

```
e.printStackTrace();
```

```
}
```

```
}
```

```
// 2. DISPLAY EMPLOYEES - READ
```

```
static void displayEmployees(Connection con) {
```

```
try {
```

```
Statement st = con.createStatement();
```

```

ResultSet rs =

    st.executeQuery("SELECT * FROM employees");

System.out.printf(

    "%-8s %-20s %-15s %-30s%n",

    "ID", "NAME", "DEPARTMENT", "EMAIL");

System.out.println(

    "-----");

while (rs.next()) {

    System.out.printf(

        "%-8d %-20s %-15s %-30s%n",

        rs.getInt("emp_id"),

        rs.getString("emp_name"),

        rs.getString("department"),

        rs.getString("email"));

    }

} catch (SQLException e) {

    e.printStackTrace();

}

}

```

```
// 3. UPDATE EMPLOYEE
```

```
static void updateEmployee(Connection con) {

    try {

        System.out.print("Enter employee ID: ");

        int id = Integer.parseInt(sc.nextLine());

        System.out.print("Enter new department: ");

        String dept = sc.nextLine();

        System.out.print("Enter new email: ");

        String email = sc.nextLine();

        PreparedStatement ps =
            con.prepareStatement(
                "UPDATE employees SET department=?, email=? " +
                "WHERE emp_id=?");

        ps.setString(1, dept);
        ps.setString(2, email);
        ps.setInt(3, id);

        int rows = ps.executeUpdate();

        if (rows > 0)

            System.out.println("Employee updated!");

        else

            System.out.println("Employee not found!");

    }
}
```

```
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
}
```

// 4. DELETE EMPLOYEE

```
static void deleteEmployee(Connection con) {  
  
    try {  
        System.out.print("Enter employee ID: ");  
        int id = Integer.parseInt(sc.nextLine());  
  
        PreparedStatement ps =  
            con.prepareStatement(  
                "DELETE FROM employees WHERE emp_id=?");  
  
        ps.setInt(1, id);  
  
        int rows = ps.executeUpdate();  
  
        if (rows > 0)  
            System.out.println("Employee deleted!");  
        else  
            System.out.println("Employee not found!");  
  
    } catch (SQLException e) {
```

```
        System.out.println(
            "Cannot delete employee because related leave records exist.");
    }
}
```

// 5. APPLY LEAVE

```
static void applyLeave(Connection con) {

    try {
        System.out.print("Enter employee ID: ");
        int empld = Integer.parseInt(sc.nextLine());

        System.out.print("Enter leave type: ");
        String type = sc.nextLine();

        System.out.print("Enter start date (YYYY-MM-DD): ");
        String start = sc.nextLine();

        System.out.print("Enter end date (YYYY-MM-DD): ");
        String end = sc.nextLine();

        System.out.print("Enter reason: ");
        String reason = sc.nextLine();

        Date startDate = Date.valueOf(start);
        Date endDate = Date.valueOf(end);
```

```
if (endDate.before(startDate)) {  
    System.out.println(  
        "End date cannot be before start date!");  
    return;  
}
```

```
PreparedStatement check =  
    con.prepareStatement(  
        "SELECT emp_id FROM employees WHERE emp_id=?");
```

```
check.setInt(1, empld);
```

```
ResultSet rs = check.executeQuery();
```

```
if (!rs.next()) {  
    System.out.println("Employee not found!");  
    return;  
}
```

```
PreparedStatement ps =  
    con.prepareStatement(  
        "INSERT INTO leave_requests " +  
        "(emp_id, leave_type, start_date, end_date, reason) " +  
        "VALUES(?,?,?,?,?)");
```

```
ps.setInt(1, empld);
```

```
ps.setString(2, type);
```

```
ps.setDate(3, startDate);
```

```
ps.setDate(4, endDate);
```

```
ps.setString(5, reason);
```

```
ps.executeUpdate();
```

```
System.out.println("Leave applied successfully!");
```

```
} catch (Exception e) {
```

```
    e.printStackTrace();
```

```
}
```

```
}
```

```
// 6. DISPLAY LEAVE REQUESTS
```

```
static void displayLeaveRequests(Connection con) {
```

```
    try {
```

```
        String sql =
```

```
            "SELECT l.leave_id, e.emp_name, l.leave_type, " +
```

```
            "l.start_date, l.end_date, l.status " +
```

```
            "FROM leave_requests l " +
```

```
            "JOIN employees e ON l.emp_id = e.emp_id";
```

```
        Statement st = con.createStatement();
```

```
        ResultSet rs = st.executeQuery(sql);
```

```
System.out.printf(
    "%-8s %-20s %-15s %-12s %-12s %-12s%n",
    "ID", "EMPLOYEE", "TYPE",
    "START", "END", "STATUS");
```

```
System.out.println(
    "-----");
```

```
while (rs.next()) {
```

```
    System.out.printf(
        "%-8d %-20s %-15s %-12s %-12s %-12s%n",
        rs.getInt("leave_id"),
        rs.getString("emp_name"),
        rs.getString("leave_type"),
        rs.getDate("start_date"),
        rs.getDate("end_date"),
        rs.getString("status"));
}
```

```
    } catch (SQLException e) {
        e.printStackTrace();
    }
```

```
}
```

```
// 7. APPROVE / REJECT - TRANSACTION
```

```

static void approveRejectLeave(Connection con) {

    try {

        System.out.print("Enter leave ID: ");

        int leaveld = Integer.parseInt(sc.nextLine());

        System.out.print("Enter A for Approve / R for Reject: ");

        String action = sc.nextLine();

        con.setAutoCommit(false);

        PreparedStatement find =
            con.prepareStatement(
                "SELECT emp_id, start_date, end_date, status " +
                "FROM leave_requests WHERE leave_id=?");

        find.setInt(1, leaveld);

        ResultSet rs = find.executeQuery();

        if (!rs.next()) {

            System.out.println("Leave request not found!");

            con.rollback();

            con.setAutoCommit(true);

            return;

        }
    }

```

```

int empld = rs.getInt("emp_id");

Date startDate = rs.getDate("start_date");
Date endDate = rs.getDate("end_date");
String oldStatus = rs.getString("status");

if (!oldStatus.equals("Pending")) {

    System.out.println("Already processed!");
    con.rollback();
    con.setAutoCommit(true);

    return;
}

// Calculate number of leave days

long difference =
    endDate.getTime() - startDate.getTime();

int leaveDays =
    (int) (difference /
    (1000 * 60 * 60 * 24)) + 1;

// APPROVE

if (action.equalsIgnoreCase("A")) {

    PreparedStatement balance =
        con.prepareStatement(

```

```
"SELECT remaining_leaves " +  
"FROM leave_balance WHERE emp_id=?");
```

```
balance.setInt(1, empId);
```

```
ResultSet br = balance.executeQuery();
```

```
if (!br.next() ||  
    br.getInt("remaining_leaves") < leaveDays) {
```

```
    System.out.println(  
        "Insufficient leave balance!");
```

```
    con.rollback();  
    con.setAutoCommit(true);  
    return;  
}
```

```
// Update leave status
```

```
PreparedStatement updateLeave =  
    con.prepareStatement(  
        "UPDATE leave_requests " +  
        "SET status='Approved' WHERE leave_id=?");
```

```
updateLeave.setInt(1, leaveId);  
updateLeave.executeUpdate();
```

```
// Update balance

PreparedStatement updateBalance =
    con.prepareStatement(
        "UPDATE leave_balance " +
        "SET used_leaves = used_leaves + ?, " +
        "remaining_leaves = remaining_leaves - ? " +
        "WHERE emp_id=?");

updateBalance.setInt(1, leaveDays);
updateBalance.setInt(2, leaveDays);
updateBalance.setInt(3, empId);

updateBalance.executeUpdate();

con.commit();

System.out.println("Leave approved!");
System.out.println(
    "Leave days: " + leaveDays);
System.out.println(
    "Transaction committed!");
}

// REJECT
```

```
else if (action.equalsIgnoreCase("R")) {
```

```
    PreparedStatement ps =  
        con.prepareStatement(  
            "UPDATE leave_requests " +  
            "SET status='Rejected' " +  
            "WHERE leave_id=?");
```

```
    ps.setInt(1, leaveId);
```

```
    ps.executeUpdate();
```

```
    con.commit();
```

```
    System.out.println("Leave rejected!");
```

```
    System.out.println(  
        "Transaction committed!");
```

```
}
```

```
// INVALID ACTION
```

```
else {
```

```
    System.out.println("Invalid action!");
```

```
    con.rollback();
```

```
}
```

```

        con.setAutoCommit(true);

    } catch (Exception e) {

        try {
            con.rollback();
            con.setAutoCommit(true);

        } catch (SQLException ex) {
            ex.printStackTrace();
        }

        System.out.println(
            "Transaction failed. Rollback completed!");
    }
}

```

// 8. LEAVE BALANCE

```

static void displayBalance(Connection con) {

    try {
        String sql =
            "SELECT e.emp_id, e.emp_name, " +
            "b.total_leaves, b.used_leaves, " +
            "b.remaining_leaves " +
            "FROM employees e " +

```

```
"JOIN leave_balance b " +  
"ON e.emp_id = b.emp_id";
```

```
Statement st = con.createStatement();
```

```
ResultSet rs = st.executeQuery(sql);
```

```
System.out.printf(  
    "%-8s %-20s %-12s %-12s %-12s%n",  
    "ID", "NAME", "TOTAL",  
    "USED", "REMAINING");
```

```
System.out.println(  
    "-----");
```

```
while (rs.next()) {
```

```
    System.out.printf(  
        "%-8d %-20s %-12d %-12d %-12d%n",  
        rs.getInt("emp_id"),  
        rs.getString("emp_name"),  
        rs.getInt("total_leaves"),  
        rs.getInt("used_leaves"),  
        rs.getInt("remaining_leaves"));
```

```
}
```

```
} catch (SQLException e) {  
    e.printStackTrace();
```

}

}

}